

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Структурная и функциональная организация вычислительных
машин

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ
на тему

Считывание, декодирование и выполнение команд. Способ адресации
операндов в командах

Выполнил
студент гр. 250541

В.Ю. Бобрик

Проверил
доцент, к.т.н. каф. ЭВМ

А.А. Воронов

Минск 2026

СОДЕРЖАНИЕ

1 Цель работы.....	3	3
2 Исходные данные к работе.....	4	3
3 Теоретические сведения.....	5	3
4 Выполнение работы.....	6	5
5 Вывод.....	11	7

1 Цель работы

Целью лабораторной работы является разработка, описание, функциональная проверка и синтез устройства «Арифметико-логическое устройство» для индивидуального варианта микро-ЭВМ. Работа выполняется так, чтобы разработанный узел мог быть использован как самостоятельный проект Quartus и как часть последующей общей структуры процессора.

В ходе выполнения необходимо определить назначение входных и выходных сигналов, разработать алгоритм функционирования устройства, описать его на языке VHDL, подготовить проект Quartus II 9.1, выполнить моделирование и подтвердить правильность работы контрольными наборами.

Таблица 1 – Этапы выполнения лабораторной работы

Этап	Содержание работы
1	Изучить индивидуальный вариант и выделить параметры устройства.
2	Разработать структурную схему, определить входные и выходные сигналы.
3	Описать функциональные блоки на языке VHDL и собрать верхний уровень проекта.
4	Подготовить проект Quartus II 9.1 и проверить корректность синтеза.
5	Разработать testbench, выполнить моделирование и сравнить результат с ожидаемым.

2 Исходные данные к работе

Лабораторная работа выполняется в среде Quartus II 9.1. Разрядность основных трактов данных принята равной 16 бит, что соответствует принятому варианту курсового проекта.

Для выполнения лабораторной работы необходимо разработать арифметико-логическое устройство и добавить его к архитектуре системы, выполненной в предыдущей лабораторной работе. Необходимо реализовать полный цикл заданных по варианту команд АЛУ. Цикл выполнения команды включает выборку команды и формирование адреса следующей команды, декодирование команды, формирование исполнительных адресов операндов, выборку операндов, выполнение операции и запись результата. Некоторые этапы цикла могут отсутствовать в зависимости от типа выполняемой команды.

В процессе выполнения команд должны использоваться специальные регистры IP, IR, AR, DR и FR.

Арифметические операции необходимо реализовать на логических элементах.

Согласно данному варианту необходимо реализовать арифметический сдвиг вправо SRA, арифметическую команду CMP с прямой регистровой адресацией, а также логическую операцию NOR. В результате выполнения операций должны формироваться признаки результата Z, S, C и O. Коды операций в проекте приняты следующими: NOR = 04h, SRA = 05h, CMP = 06h. Семейство ПЛИС для реализации — Flex10K.

3 Теоретические сведения

Алгоритм, составленный пользователем программы, в конечном счете реализуется в виде машинных команд. Под командой понимается совокупность сведений, представленных в виде двоичных кодов, которые необходимы процессору для выполнения очередного шага программы. В коде команды для сведений о типе выполняемой операции, адресной информации о расположении обрабатываемых данных, а также для информации о месте хранения результата выделяются определенные разряды, которые называются полями команды.

Множество действий, реализуемых вычислительной машиной, образует ее систему команд. Система команд во многом определяет область применения и эффективность работы вычислительной машины. Состав и количество команд должны быть ориентированы на стандартный набор операций, которые используются пользователем при решении практических задач. Чем рациональнее организована система команд, тем проще реализовать выполнение программ и тем эффективнее используется аппаратная часть устройства.

Существует несколько различных классификаций команд в вычислительных машинах. В частности, можно выделить команды пересылки данных, команды арифметической и логической обработки, команды работы со строками, команды SIMD, команды преобразования, команды ввода-вывода, команды управления системой, а также команды управления потоком команд. Каждая из этих групп предназначена для выполнения определенного класса операций и используется в зависимости от назначения вычислительной системы.

Команды пересылки данных являются одним из наиболее распространенных типов машинных команд. Они обеспечивают передачу данных между центральным процессором и оперативной памятью, между внутренними регистрами процессора, а также внутри памяти. Команда пересылки данных должна содержать информацию об адресах источника и получателя операндов, адресах ячеек памяти, номерах регистров процессора или сведения о том, что операнды находятся в стеке. Кроме того, в команде может указываться длина пересылаемых данных, заданная явно или косвенно, а также способ адресации каждого из операндов.

Команды арифметической и логической обработки обеспечивают выполнение арифметических и логических операций над информацией в

различных формах ее представления. К таким операциям относятся сложение, вычитание, сравнение, инкремент, декремент, логическое сложение, логическое умножение, инверсия и другие операции. Помимо вычисления результата, при выполнении таких команд в арифметико-логическом устройстве формируются признаки, или флаги, которые характеризуют полученный результат. К таким признакам относятся Zero, Negative, Overflow и Carry.

Команды SIMD, или Single Instruction Multiple Data, предназначены для обработки сразу нескольких групп данных одной командой. В таких командах операнды обычно представлены в упакованном формате, что позволяет выполнять однотипные операции сразу над несколькими числами. Такой подход применяется в задачах обработки массивов данных, графики, мультимедиа и других вычислений, где требуется высокая производительность при выполнении однотипных действий.

Команды работы со строками применяются для выполнения операций над последовательностями символов или байтов. Обычно к ним относятся команды перемещения, сравнения и поиска строк. В большинстве вычислительных машин такие действия могут выполняться как с помощью специальных команд, так и путем имитации строковых операций через более простые команды пересылки, сравнения и условного перехода.

Команды преобразования предназначены для изменения формы представления данных. Например, они могут использоваться для перевода чисел из одной системы счисления в другую, изменения формата данных или преобразования между различными способами кодирования. Примером такой операции является преобразование двоичного числа в двоично-десятичный код. Команды ввода-вывода обеспечивают обмен данными между процессором и внешними устройствами. Они могут быть разделены на команды управления периферийными устройствами, команды проверки состояния периферийных устройств и собственно команды ввода или вывода данных. Такие команды позволяют процессору взаимодействовать с клавиатурой, дисплеем, накопителями, портами связи и другими устройствами.

Команды управления системой относятся к привилегированным командам. Они выполняются только тогда, когда центральный процессор находится в привилегированном режиме или выполняет привилегированную программу, например операционную систему. С помощью таких команд может

изменяться состояние специальных регистров устройства управления, режимы работы процессора и другие важные параметры вычислительной системы.

Команды управления потоком команд изменяют обычный порядок выполнения программы. В стандартном случае после выполнения очередной команды счетчик команд увеличивается, и процессор переходит к следующей команде. Однако команды управления потоком могут загружать в счетчик команд новый адрес. К таким командам относятся безусловные переходы, условные переходы, вызовы процедур и возвраты из процедур. Для команд статического перехода адрес перехода может указываться через счетчик команд РС или IP. При динамическом управлении должен использоваться иной способ задания адреса перехода. Команды вызова процедуры обеспечивают переход из текущей точки программы к начальной точке процедуры, а команды возврата позволяют вернуться к команде, следующей за командой вызова. Для этого необходимо обеспечить сохранение текущего состояния счетчика команд и его последующее восстановление.

В данной лабораторной работе основное внимание уделяется командам арифметической и логической обработки, так как именно они выполняются арифметико-логическим устройством. Арифметико-логическое устройство является исполнительным узлом процессора. Оно принимает операнды из регистрового файла или памяти, выполняет выбранную операцию и передает результат в общий тракт данных. Работа АЛУ определяется кодом операции, который поступает из регистра команд и дешифрируется блоком управления. АЛУ выполняет операции над двоичными словами фиксированной разрядности. В данной работе разрядность основных трактов данных составляет 16 бит, поэтому все операции выполняются над 16-разрядными операндами. Такой подход позволяет унифицировать обработку данных и упростить построение схемы, так как все основные блоки устройства работают с одинаковой длиной слова. Результат выполнения операции также имеет разрядность 16 бит, а дополнительные сведения о результате фиксируются в регистре флагов.

Согласно варианту 4 необходимо реализовать операцию арифметического сдвига вправо SRA, арифметическую команду CMP с прямой регистровой адресацией, а также логическую операцию NOR. Эти команды относятся к разным типам операций, поэтому позволяют проверить работу АЛУ в нескольких режимах: сдвиговом, арифметическом и логическом.

Логическая операция NOR представляет собой инверсию результата операции OR. При ее выполнении сначала производится поразрядное логическое сложение двух операндов, после чего полученный результат инвертируется. Таким образом, единица в разряде результата формируется только в том случае, если оба соответствующих разряда исходных операндов равны нулю. Операция NOR относится к базовым логическим операциям и реализуется средствами комбинационной логики. В разрабатываемом устройстве она используется для получения инверсного логического результата над двумя 16-разрядными операндами.

Операция арифметического сдвига вправо SRA предназначена для сдвига двоичного слова вправо с сохранением знакового разряда. При выполнении данной операции старший бит результата получает значение старшего бита исходного слова, остальные разряды сдвигаются вправо, а младший бит исходного слова может выводиться в признак переноса C. Благодаря сохранению старшего разряда знак числа в дополнительном коде после сдвига не изменяется. Это особенно важно при обработке знаковых чисел, так как арифметический сдвиг вправо фактически соответствует делению числа на два с учетом знака.

Арифметическая команда CMP предназначена для сравнения двух операндов. В рамках варианта 4 команда CMP выполняется с прямой регистровой адресацией. Это означает, что операнды для сравнения выбираются непосредственно из регистров общего назначения по адресам, указанным в команде. При выполнении команды сравнения в АЛУ фактически анализируется разность двух операндов. Полученное значение может не записываться как обычный результат в регистр данных, однако на его основе обязательно формируются признаки состояния.

Команда CMP необходима для определения отношения между двумя операндами. С ее помощью можно установить, равны ли операнды, является ли один операнд больше или меньше другого, а также определить знак результата условного вычитания. В разрабатываемом устройстве команда CMP используется не столько для сохранения числового результата, сколько для обновления регистра флагов. Именно флаги позволяют последующим командам учитывать результат сравнения и изменять дальнейший ход выполнения программы.

После выполнения операций в арифметико-логическом устройстве формируются признаки Z, S, C и O. Признак Z устанавливается в том случае, если результат операции равен нулю. Для команды CMP этот флаг особенно важен, так как он позволяет определить равенство сравниваемых операндов. Если после условного вычитания результат равен нулю, значит оба операнда имеют одинаковое значение.

Признак S отражает знак результата и определяется по старшему разряду полученного значения. Если старший разряд результата равен единице, результат рассматривается как отрицательный в дополнительном коде. Для команды CMP данный флаг может использоваться при определении отношения между операндами, а для операции SRA он показывает сохранение знакового разряда после сдвига.

Признак C используется для фиксации переноса, заема или вытолкнутого при сдвиге бита. При выполнении операции SRA в данный флаг может помещаться младший бит исходного операнда, который был вытолкнут за пределы разрядной сетки. При выполнении команды CMP флаг C может использоваться для фиксации заема при сравнении операндов, если сравнение реализуется через операцию вычитания.

Признак O используется для фиксации арифметического переполнения. Он особенно важен при выполнении операций над знаковыми числами, когда результат выходит за пределы допустимого диапазона представления. Для логической операции NOR данный флаг обычно не имеет арифметического смысла, однако в общей структуре регистра флагов он сохраняется как один из признаков состояния АЛУ.

Таким образом, арифметико-логическое устройство не только выполняет основную операцию над операндами, но и формирует дополнительную информацию о результате. Эта информация необходима для анализа состояния вычислений, выполнения условных операций и организации дальнейшего хода программы. Поэтому правильное формирование флагов является важной частью проектирования АЛУ.

Для выполнения лабораторной работы необходимо разработать блоки, реализующие каждую заданную по варианту операцию, организовать выбор операции в зависимости от управляющих сигналов, определить источники операндов и приемники результата, а также разработать блок управления АЛУ. После этого необходимо проверить работу АЛУ как отдельного модуля, а затем

выполнить проверку всего проекта с использованием команд, записанных в памяти команд.

Таблица 2 – Характеристика команд и флагов АЛУ

Понятие	Назначение в разрабатываемом устройстве
OR	$Y = A \text{ OR } B$; используется для установки разрядов результата по единичным разрядам операндов.
NOR	$Y = \text{NOT}(A \text{ OR } B)$; позволяет получить инверсный логический результат и проверить нулевой случай.
SRA	$Y(15)=A(15)$, $Y(14..0)=A(15..1)$, $C=A(0)$; сохраняется знак числа.
INCS	$Y=A+1$ при $FR.S=1$, иначе $Y=A$; C и O формируются как для сложения с единицей.
Флаги	Z и S формируются от результата, C и O приходят из операции сдвига или сложения.

4 Выполнение работы

4.1 Структура разрабатываемого устройства

Структура проекта выбрана модульной: каждый функциональный узел вынесен в отдельную VHDL-entity, а верхний модуль соединяет эти узлы сигналами данных и управления. Такой способ удобен для проверки, так как отдельные блоки можно анализировать в RTL Viewer и независимо покрывать моделированием.

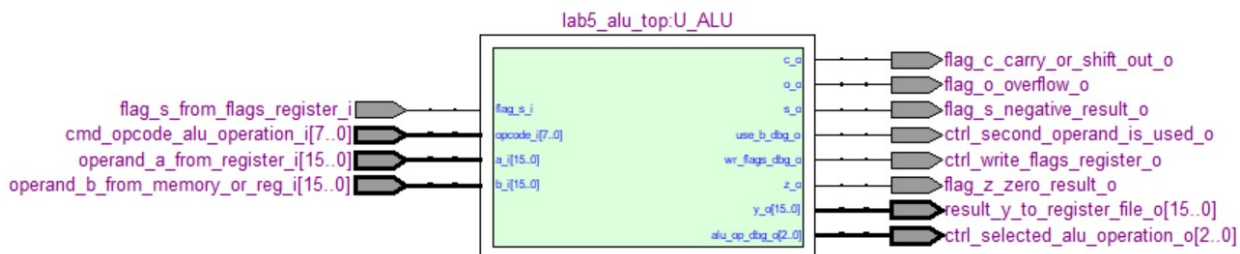


Рисунок 4.1 - Структура устройства «Арифметико-логическое устройство»

4.2 Входные и выходные сигналы

В таблице приведены основные точки наблюдения верхнего уровня. Эти же имена отображаются на схеме RTL Viewer после компиляции проекта и используются при ручном снятии временных диаграмм.

Сигнал	Направление	Разрядность	Назначение
cmd_opcode_alu_operation_i	in	8	Код операции, поступающий из регистра команды или блока управления.
operand_a_from_register_i	in	16	Первый операнд А, обычно считанный из регистра общего назначения.
operand_b_from_memory_or_reg_i	in	16	Второй операнд В для операций OR и NOR.
flag_s_from_flags_register_i	in	1	Сохраненный признак S, используемый операцией INCS.
result_y_to_register_file_o	out	16	Результат операции, передаваемый в регистровый файл или общий тракт данных.
flag_z_zero_result_o	out	1	Признак нулевого результата.
flag_s_negative_result_o	out	1	Признак отрицательного результата, равный

			старшему биту Y.
flag_c_carry_or_shift_out_o	out	1	Перенос при INCS или вытолкнутый младший бит при SRA.
flag_o_overflow_o	out	1	Признак переполнения для арифметической операции.
ctrl_selected_alu_operation_o	out	3	Диагностический код выбранной внутренней операции АЛУ.
ctrl_second_operand_is_used_o	out	1	Диагностический сигнал использования второго операнда.
ctrl_write_flags_register_o	out	1	Разрешение записи регистра флагов после выполнения операции.

4.3 Алгоритм работы

Алгоритм работы устройства задает последовательность преобразования входных сигналов в выходные и определяет условия перехода между режимами. Для синхронных блоков изменение регистров выполняется по фронту тактового сигнала, для комбинационных блоков результат формируется после изменения ВХОДОВ.

Шаг	Действие	Результат
1	Блок alu_control сравнивает код операции с OP_OR, OP_NOR, OP_SRA и OP_INCS.	Формируется внутренний код alu_op и сигнал записи флагов.
2	Блоки OR, NOR, SRA и INCS параллельно вычисляют возможные результаты.	Задержка выбора результата определяется мультиплексором, а не последовательным выполнением операций.
3	Мультиплексор выбирает один из результатов по alu_op.	На выход y_o поступает значение выбранной операции.
4	Блок alu_flags анализирует выбранный результат и служебные признаки операции.	Формируются Z, S, C и O.
5	Диагностические выходы передают код операции, использование В и разрешение записи флагов.	В RTL Viewer и на временной диаграмме видно, какая ветвь АЛУ активна.

4.4 Реализация на языке VHDL

Внутри АЛУ применена параллельная структура: каждый функциональный блок получает входные операнды одновременно. Это соответствует аппаратной реализации, в которой логические элементы OR/NOR, схема сдвига и сумматор INCS существуют как независимые ветви.

Сумматор INCS расширяет операнд до 17 бит, чтобы сохранить перенос за старший разряд. Переполнение определяется для случая прибавления единицы к положительному числу, когда старший бит результата становится единицей. Для SRA переполнение не формируется, а переносом считается младший бит исходного слова.

Формирователь флагов отделен от вычислительных блоков. Благодаря этому правила установки Z и S остаются общими для всех операций, а C и O передаются из той ветви, где они имеют смысл.

Файл	Назначение
lab_variant_pkg.vhd	Общие константы варианта: разрядность, коды операций, коды внутренних режимов АЛУ.
alu_control.vhd	Дешифрация кода операции и формирование alu_op, use_b, write_flags.
alu_or.vhd	Комбинационный блок поразрядной операции OR.
alu_nor.vhd	Комбинационный блок поразрядной операции NOR.
alu_sra.vhd	Арифметический сдвиг вправо с выдачей вытолкнутого бита в C.
alu_incs.vhd	Условный инкремент по флагу S с расчетом переноса и переполнения.
alu_flags.vhd	Формирователь признаков результата Z, S, C и O.
lab5_alu_top.vhd	Структурное объединение всех блоков АЛУ.
lab5_alu_rtl_view_top.vhd	Верхний уровень с понятными именами пинов для RTL Viewer.
tb_lab5_alu.vhd	Тестовая модель с assert-проверками всех операций.

Файл проекта Quartus: lab5_alu/quartus/lab5_alu.qpf. Верхний уровень проекта: lab5_alu_rtl_view_top. После открытия проекта необходимо выполнить Processing -> Start Compilation, затем открыть Tools -> Netlist Viewers -> RTL Viewer для просмотра схемы.

4.5 Функциональное моделирование

Функциональная проверка выполнена testbench-моделью. В testbench подаются контрольные наборы, соответствующие рабочим и граничным режимам устройства. Проверка результата выполняется операторами assert, поэтому ошибка в данных, флагах или протоколе приводит к останову моделирования.

Проверка	Условие	Ожидаемый результат
OR	A=00F0h, B=0F0Fh, opcode=03h	Y=0FFFh, Z=0, S=0
NOR	A=0FFFh, B=0FFFh, opcode=04h	Y=F000h, S=1
SRA	A=8001h, opcode=05h	Y=C000h, C=1, знак сохранен
INCS	A=C000h, FR.S=1, opcode=06h	Y=C001h
INCS без инкремента	A=1234h, FR.S=0	Y=1234h

Нулевой результат	A=FFFFh, B=FFFFh, opcode=04h	Y=0000h, Z=1
-------------------	------------------------------	--------------

ЛР5. Контрольные операции АЛУ

t, ns	0	10	20	30	40	50
opcode	OR	NOR	SRA	INCS	INCS	NOR
A	00F0	0FFF	8001	C000	1234	FFFF
B	0F0F	0FFF	FFFF	FFFF	FFFF	FFFF
FR.S	0	0	0	1	0	0
Y	0FFF	F000	C000	C001	1234	0000
Z S C O	0000	0100	0110	0100	0000	1000

Рисунок 4.2 - Контрольная временная диаграмма для ЛР5

4.6 Результаты синтеза

Компиляция проекта выполняется в Quartus II 9.1. VHDL-описание не использует вручную размещенные вентиляльные примитивы, поэтому схема синтезируется средствами Quartus на выбранную элементную базу. Основные показатели компиляции приведены в таблице.

Показатель	Значение
Combinational ALUTs	90
Dedicated registers	0
Pins	66
Block memory bits	0
Результат компиляции	Full compilation: 0 errors

5 Вывод

В ходе лабораторной работы разработано арифметико-логическое устройство для операций OR, NOR, SRA и INCS. Реализация разбита на независимые блоки вычисления, управления и формирования флагов, что упрощает анализ схемы и соответствует аппаратной структуре АЛУ.

Моделирование подтвердило корректность всех операций индивидуального варианта: логические операции дают ожидаемые поразрядные результаты, арифметический сдвиг сохраняет знак и формирует перенос, а INCS изменяет операнд только при установленном флаге S. Проект успешно компилируется в Quartus и готов для использования в составе микро-ЭВМ.